

UPI Failed Debit — Beneficiary Not Credited

Document control

- Requirement ID: `upi_failed_debit_no_credit.requirements.v2`
- Intended application ID: `upi_failed_debit_no_credit`
- Human-readable application name: `UPI Failed Debit – Beneficiary Not Credited`
- Scenario class: Payment operations dispute and exception case management
- Engineering-depth profile: advanced governed reference application
- Execution posture: local, deterministic-first, mock-safe, evidence-driven, human-gated
- Data posture: fictional test data only
- External-integration posture: local deterministic mocks only
- Production posture: not production-ready
- Certification posture: certification-ready-not-certified
- Real payment calls: prohibited
- Live provider calls: prohibited
- Default LLM calls: zero
- Requirement conformance policy: fail closed
- Unsupported mandatory capability policy: fail closed with explicit evidence
- Traceability policy: every mandatory requirement must map to implementation, tests and evidence

1. Governing generation contract

The factory shall treat every statement containing **shall, must, required, mandatory, prohibited, or fail closed** as an enforceable engineering obligation.

The generated application shall not be considered complete merely because source files were produced.

Completion requires all of the following:

1. Every mandatory requirement is represented in a requirements-traceability matrix.
2. Every mandatory domain rule maps to at least one implementation location and one executable test.
3. Every mandatory API maps to an OpenAPI operation and an executable contract test.
4. Every mandatory state transition maps to implementation and positive and negative tests.
5. Every mandatory safety boundary maps to an enforcement mechanism and test evidence.
6. Generated tests are actually executed.
7. Ruff, MyPy, Python compilation and runtime probes pass.
8. The generated application starts locally and exposes health, readiness and OpenAPI.
9. All evidence artifacts are checksummed.
10. Any unsupported mandatory obligation causes a governed `NO_GO` result.

The factory shall not silently omit, weaken, reinterpret or mark unsupported mandatory requirements as satisfied.

The generated evidence shall explicitly distinguish:

- implemented and verified obligations;
- implemented but partially verified obligations;
- unsupported obligations;
- deferred obligations;
- residual risks;
- non-goals.

Any unsupported or partially verified mandatory obligation shall prevent a final **GO** result.

2. Problem statement

A payer reports that an account was debited for a UPI transaction, but the beneficiary did not receive the funds.

Operations teams require a controlled dispute workflow that:

- preserves transaction and investigation evidence;
- prevents duplicate case creation and duplicate compensation;
- distinguishes debit, beneficiary credit, reversal and unknown conditions;
- records deterministic classifications and reason codes;
- escalates uncertainty and high-impact cases;
- enforces human accountability for consequential decisions;
- provides traceable case history;
- produces independently verifiable engineering and operational evidence.

The application shall demonstrate a production-shaped architecture without performing real payment processing, account actions, regulatory filings or live provider interactions.

3. Business objective

Engineer a locally runnable reference application that demonstrates:

- governed complaint intake;
- deterministic input validation;
- idempotent case creation;
- domain-driven dispute modelling;
- explicit state-machine enforcement;
- mock transaction-status investigation;
- evidence completeness assessment;
- deterministic classification;
- confidence and impact assessment;
- human-review routing;
- controlled resolution and closure;
- append-only audit history;
- structured APIs and stable error contracts;

- SQLite-backed restart persistence;
- structured logs and metrics;
- OpenAPI publication;
- generated and executed tests;
- requirements-to-code-to-test traceability;
- checksummed evidence and downloadable artifacts.

The application shall remain:

- local-only;
- deterministic-first;
- mock-only;
- replaceable;
- independently testable;
- free from hidden network dependencies;
- certification-ready-not-certified;
- explicitly not production-ready.

4. Users, actors and accountability

The application shall recognise the following fictional actor roles:

- PAYER
- CUSTOMER_SUPPORT_AGENT
- DISPUTE_OPERATIONS_ANALYST
- SUPERVISOR_APPROVER
- AUDIT_REVIEWER
- SYSTEM MOCK_ADAPTER

The application shall enforce role-based action permissions.

4.1 Role permissions

A payer may:

- submit a fictional complaint;
- inquire about a case using the generated case identifier.

A customer-support agent may:

- create a case on behalf of a payer;
- add non-consequential evidence observations;
- view case status.

A dispute-operations analyst may:

- request mock transaction status;
- assess evidence;

- propose a classification;
- request human review.

A supervisor approver may:

- approve or reject a proposed consequential disposition;
- authorise case closure;
- record a reason code and rationale.

An audit reviewer may:

- inspect case history;
- inspect evidence metadata;
- inspect decision records;
- inspect audit-integrity results;
- not alter operational case state.

A mock adapter may:

- supply deterministic fictional transaction observations;
- never make an external call;
- never make a consequential case decision.

4.2 Human-accountability rule

All final consequential classifications, disposition decisions and closure decisions shall remain attributable to an authorised human actor.

The system may recommend or propose a classification but shall not autonomously:

- credit a payer;
- debit a beneficiary;
- reverse a payment;
- submit a network instruction;
- restrict an account;
- file a regulatory report;
- certify a dispute outcome;
- close a high-impact or uncertain case.

5. Domain glossary

The generated application shall use the following domain terms consistently.

5.1 Dispute case

A governed record representing one fictional failed-debit or beneficiary-not-credited complaint.

5.2 Transaction observation

A deterministic mock observation describing debit, beneficiary-credit, reversal and provider-status signals.

5.3 Evidence item

A metadata record referring to fictional investigation information. Raw secret material or real personal data shall not be stored.

5.4 Classification

A deterministic interpretation of available transaction observations and evidence.

Supported classifications:

- PENDING
- FAILED
- REVERSED
- CREDIT_CONFIRMED
- UNCERTAIN

5.5 Human review

A governed activity requiring an authorised reviewer to accept, reject or modify a proposed disposition.

5.6 Disposition

The recorded operational conclusion of the case. A disposition is not a payment instruction.

Supported dispositions:

- WAIT_FOR_NETWORK_UPDATE
- CONFIRM_FAILURE_FOR_MANUAL_FOLLOW_UP
- CONFIRM_REVERSAL_OBSERVED
- CONFIRM_BENEFICIARY_CREDIT_OBSERVED
- REQUIRE_ADDITIONAL_EVIDENCE
- ESCALATE_FOR_SPECIALIST_REVIEW
- UNRESOLVED

6. Required domain model

The application shall implement logically separated domain objects rather than storing the entire case as an unstructured dictionary.

6.1 Aggregate root: DisputeCase

The DisputeCase aggregate shall include:

- case_id
- transaction_reference
- idempotency_key
- request_payload_digest
- amount_minor
- currency
- complaint_summary
- case_state
- classification
- classification_reason_code
- classification_confidence
- impact_level
- evidence_completeness
- human_review_required
- human_review_status
- proposed_disposition
- approved_disposition
- created_by
- created_at
- updated_at
- version
- correlation_id

The aggregate shall own and enforce its state transitions and invariants.

6.2 Required value objects

Implement explicit value objects for:

- CaseId
- TransactionReference
- IdempotencyKey
- Money
- CorrelationId
- ActorId
- EvidenceDigest
- ReasonCode
- ConfidenceScore
- CaseVersion

Value objects shall validate their own invariants and shall be immutable where practical.

6.3 Required supporting entities

Implement:

- TransactionObservation
- EvidenceItem
- ClassificationDecision
- HumanReviewDecision
- AuditEvent
- IdempotencyRecord

6.4 Domain invariants

The domain layer shall enforce at least these invariants:

1. Amount must be greater than zero.
2. Currency must be an explicitly supported fictional demonstration currency.
3. Transaction reference must be present and structurally valid.
4. A case cannot close without a recorded disposition.
5. A case requiring human review cannot close without an approved human-review decision.
6. A rejected human-review decision cannot be treated as approval.
7. An uncertain classification always requires human review.
8. Contradictory evidence always requires human review.
9. A high-impact case always requires human review.
10. Audit history cannot be replaced or reordered through an application service.
11. An idempotency key cannot be associated with two different payload digests.
12. A closed case cannot be modified except by an explicitly modelled administrative correction workflow.
13. A classification must include a reason code.
14. A human decision must include actor, timestamp, rationale and correlation ID.
15. A disposition shall not represent or execute an actual financial action.

7. Case state machine

The application shall implement an explicit state machine.

Required states:

- RECEIVED
- VALIDATED
- INVESTIGATING
- AWAITING_EVIDENCE
- AWAITING_HUMAN_REVIEW
- DECISION_RECORDED
- RESOLVED
- CLOSED
- QUARANTINED

7.1 Allowed transitions

At minimum, support:

- RECEIVED -> VALIDATED
- VALIDATED -> INVESTIGATING
- INVESTIGATING -> AWAITING_EVIDENCE
- INVESTIGATING -> AWAITING_HUMAN_REVIEW
- INVESTIGATING -> DECISION_RECORDED
- AWAITING_EVIDENCE -> INVESTIGATING
- AWAITING_EVIDENCE -> AWAITING_HUMAN_REVIEW
- AWAITING_HUMAN_REVIEW -> DECISION_RECORDED
- DECISION_RECORDED -> RESOLVED
- RESOLVED -> CLOSED
- any non-closed state to QUARANTINED when integrity or policy enforcement fails.

7.2 Prohibited transitions

At minimum, reject:

- RECEIVED -> CLOSED
- VALIDATED -> CLOSED
- INVESTIGATING -> CLOSED
- AWAITING_EVIDENCE -> CLOSED
- AWAITING_HUMAN_REVIEW -> CLOSED
- CLOSED -> INVESTIGATING
- QUARANTINED -> CLOSED without an explicit governed recovery decision.

Every prohibited transition shall return a structured policy error and shall create a redacted audit event.

8. Input contract

The application shall accept only fictional demonstration data.

8.1 Required complaint fields

- client-supplied idempotency key;
- fictional transaction reference;
- amount in minor currency units;
- supported currency code;
- complaint summary;
- actor ID;
- actor role;
- correlation ID.

8.2 Optional fields

- fictional payer reference;
- fictional beneficiary reference;
- transaction timestamp;
- evidence observations;
- mock fault profile;
- requested review priority.

8.3 Validation requirements

Reject:

- missing mandatory values;
- malformed identifiers;
- zero or negative amounts;
- unsupported currencies;
- oversized text;
- unsupported role;
- unsupported state transition;
- malformed correlation ID;
- duplicate evidence identifier;
- invalid confidence value;
- invalid reason code;
- unsupported disposition;
- path traversal content;
- obvious shell or command injection content;
- obvious credential, password, token, PIN or API-key material;
- real-person or real-account patterns where detectable;
- unsupported production, certification or regulatory claims.

Validation failures shall not create a partially persisted dispute case unless the failure itself is intentionally recorded in an isolated, redacted rejection audit store.

9. Idempotency and concurrency

9.1 Equivalent replay

A repeated request with the same idempotency key and an equivalent canonical payload shall:

- return the original case identifier;
- return the original creation result;
- not create a second case;
- record an idempotent replay audit event;
- produce a stable response contract.

9.2 Conflicting replay

Reuse of an idempotency key with a different canonical payload digest shall:

- fail closed;
- return an `IDEMPOTENCY_CONFLICT` error;
- not alter the original case;
- not create a second case;
- record a redacted conflict audit event.

9.3 Concurrent replay

Two concurrent equivalent requests using the same idempotency key shall result in exactly one case.

The generated tests shall exercise concurrent or near-concurrent duplicate submission using deterministic local execution.

9.4 Optimistic concurrency

Case-changing commands shall carry or enforce an expected case version.

A stale update shall return a structured `CONCURRENCY_CONFLICT` response and shall not overwrite a newer case state.

10. Mock transaction-status integration

Implement a port named conceptually:

`TransactionStatusPort`

Provide a local deterministic adapter.

The adapter shall accept a fictional transaction reference and return a `TransactionObservation` containing:

- debit status;
- beneficiary-credit status;
- reversal status;
- observation timestamp;
- deterministic provider reason code;
- mock adapter version;
- observation digest.

Supported mock statuses shall include:

- debit succeeded, credit pending;
- debit succeeded, credit failed;

- debit succeeded, reversal confirmed;
- debit succeeded, beneficiary credit confirmed;
- contradictory status;
- unknown status;
- simulated timeout;
- simulated temporary read failure.

The adapter shall:

- perform no network access;
- use no real credentials;
- return deterministic results for a given test fixture;
- support bounded fault injection;
- record invocation evidence without logging complete request bodies.

11. Deterministic classification rules

The application shall implement explicit classification rules.

At minimum:

1. Debit success plus beneficiary credit pending plus no reversal:
2. classification: `PENDING`
3. reason code: `CREDIT_PENDING`
4. disposition proposal: `WAIT_FOR_NETWORK_UPDATE`
5. Debit success plus beneficiary credit failed plus no reversal:
6. classification: `FAILED`
7. reason code: `BENEFICIARY_CREDIT_FAILED`
8. human review required when impact is high or evidence is incomplete.
9. Debit success plus reversal confirmed:
10. classification: `REVERSED`
11. reason code: `REVERSAL_CONFIRMED`
12. Debit success plus beneficiary credit confirmed:
13. classification: `CREDIT_CONFIRMED`

14. reason code: BENEFICIARY_CREDIT_CONFIRMED

15. Unknown, missing or contradictory observations:

16. classification: UNCERTAIN

17. reason code: INSUFFICIENT_OR_CONTRADICTIONARY_EVIDENCE

18. human review required.

Classification rules shall be represented in code, tests and a generated decision table.

12. Confidence, impact and escalation

12.1 Confidence

Use a deterministic confidence score from 0 to 100.

The score shall be derived from documented rule inputs, not an opaque model.

At minimum, confidence shall consider:

- completeness of debit observation;
- completeness of beneficiary-credit observation;
- completeness of reversal observation;
- evidence consistency;
- observation freshness;
- adapter response integrity.

12.2 Impact

Implement impact levels:

- LOW
- MEDIUM
- HIGH
- CRITICAL

The default fictional high-value threshold shall be configurable and shall default to 100000 minor currency units.

12.3 Mandatory escalation

Human review is mandatory when:

- classification is UNCERTAIN ;
- confidence is below the configured threshold;
- impact is HIGH or CRITICAL ;

- evidence is contradictory;
- evidence completeness is insufficient;
- a stale case version is detected;
- audit-integrity verification fails;
- an unsupported disposition is proposed;
- a policy-refusal condition occurs.

13. Evidence management

Evidence items shall store metadata only.

Each evidence item shall include:

- evidence ID;
- case ID;
- evidence type;
- fictional source;
- redacted description;
- SHA-256 digest;
- actor ID;
- correlation ID;
- creation timestamp;
- classification relevance;
- integrity status.

The application shall not store:

- real account numbers;
- full card numbers;
- UPI PINs;
- passwords;
- access tokens;
- API keys;
- authentication secrets;
- real customer names or addresses;
- raw unredacted request bodies.

Evidence completeness shall be reported as:

- COMPLETE
- PARTIAL
- CONTRADICTORY
- MISSING

14. Human-review workflow

The application shall support:

1. request human review;
2. retrieve pending reviews;
3. record reviewer decision;
4. approve, reject or return for more evidence;
5. record rationale and reason code;
6. prevent the reviewer from approving their own prohibited action when segregation-of-duties rules apply;
7. preserve review history.

Required review statuses:

- NOT_REQUIRED
- PENDING
- APPROVED
- REJECTED
- MORE_EVIDENCE_REQUIRED

A review decision shall require:

- authorised actor role;
- case version;
- decision;
- reason code;
- rationale;
- correlation ID.

15. Required application use cases

Implement separate application services or command handlers for:

- create dispute case;
- retrieve dispute case;
- search by fictional transaction reference;
- add evidence metadata;
- request transaction observation;
- classify case;
- request human review;
- record human-review decision;
- record disposition;
- close case;
- quarantine case;
- retrieve case history;
- verify audit integrity;

- reset local demonstration data through an explicitly governed operation.

Use cases shall depend on ports, not concrete infrastructure classes.

16. Required ports and interfaces

Define ports for:

- dispute-case repository;
- idempotency repository;
- transaction-status adapter;
- evidence repository;
- audit-event repository;
- human-review repository;
- clock;
- identifier generator;
- configuration provider;
- metrics recorder;
- notification mock;
- unit of work or transaction boundary.

Infrastructure adapters shall be replaceable without changing domain rules.

17. Persistence and transaction boundaries

Provide a SQLite-backed local adapter.

Requirements:

- explicit schema or migrations;
- transaction boundaries around state-changing use cases;
- durable idempotency records;
- durable case state;
- durable audit events;
- durable human-review decisions;
- restart persistence;
- deterministic reset command;
- repository interfaces separated from SQL details;
- no hidden external database dependency.

The application may additionally provide an in-memory adapter for isolated unit tests.

The SQLite database shall remain inside the selected local state root.

18. Audit architecture

Audit events shall be append-only through normal application operations.

Each audit event shall include:

- event ID;
- sequence number;
- case ID;
- event type;
- actor ID;
- actor role;
- action;
- reason code;
- redacted details;
- correlation ID;
- timestamp;
- previous-event digest;
- current-event digest.

The application shall verify the audit digest chain.

Tampering, missing sequence numbers or invalid digest links shall:

- fail audit verification;
- create or request quarantine;
- prevent normal closure;
- produce explicit evidence.

No application service shall support arbitrary audit-event deletion or replacement.

19. API contract

Use FastAPI with typed request and response models.

19.1 Platform endpoints

Required:

- GET /health
- GET /ready
- GET /openapi.json
- GET /docs
- GET /metrics or an equivalent local machine-readable metrics endpoint.

19.2 Case endpoints

Required:

- POST /v1/disputes
- GET /v1/disputes/{case_id}

- GET /v1/disputes/{case_id}/history
- GET /v1/disputes?transaction_reference={reference}
- POST /v1/disputes/{case_id}/evidence
- POST /v1/disputes/{case_id}/investigate
- POST /v1/disputes/{case_id}/classify
- POST /v1/disputes/{case_id}/human-review
- POST /v1/disputes/{case_id}/review-decisions
- POST /v1/disputes/{case_id}/disposition
- POST /v1/disputes/{case_id}/close
- POST /v1/disputes/{case_id}/quarantine
- GET /v1/disputes/{case_id}/audit-integrity

19.3 API headers

State-changing operations shall support or require:

- Idempotency-Key where applicable;
- X-Correlation-ID ;
- fictional actor identity;
- fictional actor role;
- expected case version where applicable.

19.4 Structured errors

Errors shall use a stable machine-readable envelope containing:

- error code;
- message;
- correlation ID;
- retryable flag;
- details safe for disclosure.

Required error categories:

- VALIDATION_FAILED
- NOT_FOUND
- IDEMPOTENCY_CONFLICT
- CONCURRENCY_CONFLICT
- UNAUTHORISED_ROLE
- INVALID_STATE_TRANSITION
- POLICY_REFUSAL
- EVIDENCE_INCOMPLETE
- HUMAN_REVIEW_REQUIRED
- AUDIT_INTEGRITY_FAILED
- MOCK_ADAPTER_FAILURE
- INTERNAL_INCONSISTENCY

Internal stack traces shall not be returned to API clients.

20. Security and privacy controls

The generated application shall implement:

- strict typed validation;
- bounded string lengths;
- identifier allowlists;
- enum validation;
- path traversal rejection;
- redaction before logging;
- secret-pattern detection;
- role enforcement;
- correlation-ID propagation;
- explicit local-only host binding;
- prohibited-network enforcement;
- dependency inventory;
- secure default configuration;
- no hard-coded secrets;
- no real credentials;
- no dynamic arbitrary shell execution;
- no untrusted code evaluation.

The application shall bind to loopback by default.

21. Reliability requirements

- Deterministic startup and shutdown.
- Idempotent shutdown.
- Health and readiness probes.
- SQLite restart persistence.
- Explicit exception handling.
- No silent exception suppression.
- No silent retry of consequential commands.
- Bounded retries only for safe mock reads.
- Retry count and delay must be configurable and bounded.
- Mock timeout handling.
- Transaction rollback on failed state-changing operations.
- Stale-version rejection.
- Stable machine-readable terminal status.
- Temporary files shall remain under the configured state root or `/tmp`.
- No generated runtime file shall alter tracked source files.

22. Observability

22.1 Structured logs

Emit structured JSON logs containing:

- timestamp;
- severity;
- event name;
- case ID where applicable;
- app ID;
- version ID where available;
- actor ID;
- correlation ID;
- outcome;
- duration;
- reason code.

Logs shall be redacted before persistence.

22.2 Metrics

Provide local metrics for:

- cases created;
- idempotent replays;
- idempotency conflicts;
- validation failures;
- classifications by type;
- human reviews requested;
- review approvals and rejections;
- invalid transitions;
- audit-integrity failures;
- mock-adapter failures;
- API latency;
- active cases by state.

Metrics shall not contain real or fictional personal identifiers.

22.3 Traceability

Correlation IDs shall propagate across:

- API request;
- application service;
- mock adapter;
- persistence operation;

- audit event;
- evidence record;
- response.

23. Configuration

Provide documented configuration for:

- local host;
- local port;
- state root;
- SQLite path;
- fictional high-value threshold;
- confidence escalation threshold;
- safe mock retry count;
- safe mock retry delay;
- log level;
- data-retention period;
- reset enablement;
- mock scenario profile.

Configuration shall fail closed for unsupported or unsafe values.

No OpenAI API key shall be required for the deterministic execution path.

24. Required generated documentation

Generate:

- `README.md`
- architecture overview;
- domain model description;
- state-transition table;
- classification decision table;
- API usage guide;
- local startup and shutdown guide;
- operations runbook;
- troubleshooting guide;
- threat model;
- test plan;
- requirements-traceability matrix;
- residual-risk register;
- mock-ecosystem boundary statement;
- certification-ready-not-certified boundary statement;
- architecture decision records for significant design choices;
- persistence and reset policy;
- evidence-manifest description.

Documentation shall match executable commands and generated implementation.

25. Packaging and local operation

The generated application shall include:

- a documented local startup command;
- a documented shutdown command;
- a local health probe;
- a local readiness probe;
- Dockerfile where supported;
- Docker Compose configuration where supported;
- persistent local state configuration;
- non-root container execution where practical;
- loopback-only host publication;
- mock-only environment defaults;
- LLM-disabled default configuration.

No packaging artifact shall claim production readiness.

26. Testing obligations

The generated application shall be subjected to all applicable tests below.

26.1 Compilation and static analysis

- Python compilation.
- Ruff with no unresolved violations.
- MyPy with no unresolved generated-package errors.
- Import and package validation.
- OpenAPI generation validation.

26.2 Domain tests

- value-object validation;
- aggregate invariants;
- every allowed state transition;
- every prohibited state transition;
- deterministic classification table;
- confidence calculation;
- impact calculation;
- mandatory escalation;
- closure prerequisites;
- quarantine behaviour;
- audit-digest-chain validation.

26.3 Application-service tests

- create case;
- retrieve case;
- add evidence;
- investigate;
- classify;
- request review;
- record review decision;
- record disposition;
- close;
- quarantine;
- reset;
- persistence restart;
- transaction rollback.

26.4 Idempotency and concurrency tests

- equivalent replay returns the original case;
- conflicting replay fails closed;
- concurrent equivalent replay creates one case;
- stale version fails;
- valid expected version succeeds.

26.5 API tests

- positive contract tests;
- malformed request tests;
- missing-header tests;
- wrong-role tests;
- not-found tests;
- idempotency-conflict tests;
- concurrency-conflict tests;
- invalid-transition tests;
- human-review-required tests;
- audit-integrity-failure tests;
- stable error-envelope tests;
- OpenAPI path and schema tests.

26.6 Security and boundary tests

- PII-pattern rejection or redaction;
- secret-pattern rejection;
- no complete request-body logging;
- no hidden network access;
- no live provider calls;
- no real payment calls;
- no arbitrary shell execution;

- path traversal rejection;
- unsafe configuration rejection;
- prohibited production and certification claim rejection.

26.7 Persistence tests

- SQLite schema creation;
- migration application;
- create and retrieve after restart;
- durable idempotency;
- durable audit history;
- rollback behaviour;
- deterministic reset;
- state-root containment.

26.8 Observability tests

- structured log schema;
- correlation propagation;
- metrics availability;
- metrics update after operations;
- redaction;
- runtime health;
- readiness;
- latency probe recording.

26.9 Packaging tests

- startup from a clean checkout;
- local shutdown;
- repeated shutdown;
- Docker build where applicable;
- Docker health where applicable;
- application download;
- evidence download;
- package checksum verification;
- archive path safety;
- tracked-source cleanliness after runtime execution.

26.10 Coverage expectations

The generated evidence shall report statement and branch coverage when the configured test tooling supports it.

Target thresholds:

- statement coverage: at least 90%;
- branch coverage: at least 85%;

- domain layer: at least 95% statement coverage.

If coverage tooling is unsupported, the factory shall report the obligation as unsupported and shall not silently claim it passed.

27. Mandatory acceptance scenarios

Scenario A — Successful case creation

Given valid fictional complaint data,
when a case is created,
then a unique case ID is returned,
the case is persisted,
an audit event is recorded,
and the state is `RECEIVED` or the next valid governed state.

Scenario B — Equivalent idempotent replay

Given an existing request,
when the same idempotency key and equivalent payload are submitted,
then the original case is returned and no duplicate is created.

Scenario C — Conflicting idempotent replay

Given an existing request,
when the same idempotency key is used with a different payload,
then the request fails with `IDEMPOTENCY_CONFLICT`.

Scenario D — Pending beneficiary credit

Given debit success, credit pending and no reversal,
when the case is investigated,
then classification is `PENDING`.

Scenario E — Credit failure

Given debit success and beneficiary-credit failure,
when the case is investigated,
then classification is `FAILED`.

Scenario F — Reversal confirmed

Given debit success and confirmed reversal,
when the case is investigated,
then classification is `REVERSED`.

Scenario G — Credit confirmed

Given debit success and confirmed beneficiary credit,
when the case is investigated,
then classification is `CREDIT_CONFIRMED`.

Scenario H — Contradictory evidence

Given contradictory transaction observations,
when classification is attempted,
then classification is `UNCERTAIN` and human review is required.

Scenario I — High-impact case

Given an amount at or above the configured high-value threshold,
when classification is proposed,
then human review is required.

Scenario J — Missing evidence

Given incomplete required observations,
when resolution is attempted,
then the case cannot close and more evidence or human review is required.

Scenario K — Invalid transition

Given a case in `RECEIVED`,
when direct closure is attempted,
then the operation fails with `INVALID_STATE_TRANSITION`.

Scenario L — Unauthorised actor

Given a customer-support actor,
when a supervisor-only decision is attempted,
then the operation fails with `UNAUTHORISED_ROLE`.

Scenario M — Restart persistence

Given a persisted case,
when the local application restarts,
then the case, idempotency record, audit events and review decisions remain available.

Scenario N — Audit tampering

Given an altered audit record,
when audit integrity is verified,
then verification fails and the case is quarantined or blocked from closure.

Scenario O — Secret-pattern submission

Given input containing an apparent password, token, PIN or API key,
when submitted,
then the request is rejected or safely redacted according to policy and no secret appears in logs.

Scenario P — Unsupported production claim

Given a request to certify or declare the application production-ready,
when processed,
then the request is rejected with `POLICY_REFUSAL`.

28. Evidence obligations

The run shall produce:

- exact requirements copy;
- requirements SHA-256;
- normalized requirement model where supported;
- implementation inventory;
- generated-file manifest;
- requirements-traceability matrix;
- domain-rule-to-test matrix;
- API-to-test matrix;
- state-transition test matrix;
- test command records;
- test exit codes;
- test counts;
- Ruff result;
- MyPy result;
- Python compilation result;
- coverage result or explicit unsupported status;
- runtime health result;
- readiness result;
- OpenAPI document;
- OpenAPI inventory;
- OpenAPI SHA-256;
- application identity;
- version identity;
- mock-boundary evidence;
- prohibited-network evidence;
- secret-leakage test evidence;
- package manifest;
- application archive checksum;
- evidence archive checksum;
- residual-risk register;
- unsupported-obligation report;

- final `GO` or `NO_GO` decision.

Evidence shall be linked to:

- requirement ID;
- requirements hash;
- application ID;
- version ID;
- run ID;
- source factory commit where available.

29. Final acceptance criteria

The generated application is accepted only when:

1. The complete requirement is traceable.
2. All mandatory obligations are implemented or the run is explicitly `NO_GO`.
3. The dispute aggregate and explicit state machine exist.
4. Idempotent creation and conflict detection work.
5. Concurrent duplicate submission creates one case.
6. SQLite restart persistence works.
7. Deterministic mock investigation works.
8. All supported classifications are tested.
9. Uncertain and high-impact cases require human review.
10. Consequential closure requires an authorised human decision.
11. Audit history is append-only through normal operations.
12. Audit-chain verification detects tampering.
13. Structured APIs and error contracts are stable.
14. Health, readiness, metrics, OpenAPI and documentation are available.
15. Generated tests were actually executed.
16. Static analysis passed.
17. Runtime probes passed.
18. No hidden network or real-provider access occurred.
19. No real payment action occurred.
20. No secret or real-person data appears in logs or evidence.
21. Application and evidence downloads succeed.
22. Checksums match.
23. Runtime execution does not modify tracked source.
24. Limitations and residual risks are explicitly documented.
25. The final posture remains certification-ready-not-certified and not production-ready.

30. Fail-closed conditions

The run shall fail closed when:

- approval is absent or incorrect;
- the requirements file is missing, unreadable or malformed;

- the application ID is invalid;
- mandatory requirement traceability is incomplete;
- a mandatory requirement is silently omitted;
- output or evidence escapes the isolated workspace or `/tmp`;
- live-payment or provider access is requested;
- real secrets or real-person data are detected;
- idempotency invariants fail;
- concurrent duplicate creation occurs;
- state-machine invariants fail;
- audit integrity fails without quarantine;
- human-review controls are bypassed;
- generated tests are not executed;
- Python compilation fails;
- Ruff fails;
- MyPy fails;
- required runtime probes fail;
- OpenAPI cannot be generated;
- application or evidence packaging fails;
- evidence checksums do not match;
- tracked source is mutated by runtime execution;
- unsupported production-readiness or certification claims are attempted;
- any mandatory unsupported obligation is incorrectly reported as passed.

31. Non-goals and limitations

- No real payment processing.
- No automated refund.
- No automated reversal.
- No real debit or credit.
- No account restriction.
- No real customer, merchant, bank, PSP, NPCI, issuer or acquirer data.
- No live UPI or banking integration.
- No regulatory filing.
- No legal or regulatory determination.
- No official certification.
- No production deployment.
- No claim of production readiness.
- No claim that deterministic classification replaces human accountability.
- No guarantee that every obligation can be implemented by the current deterministic generator.

The current deterministic factory may still use a common production-shaped architecture across application IDs.

This requirement intentionally increases scenario-specific depth through:

- an explicit domain aggregate;
- value objects;

- state transitions;
- deterministic classification;
- confidence and impact rules;
- human review;
- persistence;
- concurrency;
- audit integrity;
- richer APIs;
- expanded tests;
- stronger evidence.

If the current generator cannot implement one or more mandatory depth requirements, it shall return `NO_GO` with an unsupported-capability report rather than generating a shallow application and declaring success.

32. Official-reference boundary

Use the official-reference index at:

`docs/OFFICIAL_REFERENCES.md`

for domain orientation only.

Before any real-world implementation, production use, regulatory reliance or external integration:

- current obligations must be independently verified;
- applicable NPCI, RBI, bank, PSP and organisational requirements must be confirmed;
- security, privacy, operational and legal reviews must be completed;
- official certification or approval must be obtained where applicable.

This local reference application and its evidence do not constitute regulatory approval, production readiness or official certification.